

Accepted Manuscript

Optimizing cloud solutioning design

Aly Megahed, Ahmed Nazeem, Peifeng Yin, Samir Tata, Hamid Reza
Motahari Nezhad, Taiga Nakamura

PII: S0167-739X(18)30615-0
DOI: <https://doi.org/10.1016/j.future.2018.08.005>
Reference: FUTURE 4389

To appear in: *Future Generation Computer Systems*

Received date: 20 March 2018

Revised date: 30 June 2018

Accepted date: 3 August 2018

Please cite this article as:, Optimizing cloud solutioning design, *Future Generation Computer Systems* (2018), <https://doi.org/10.1016/j.future.2018.08.005>

This is a PDF file of an unedited manuscript that has been accepted for publication. As a service to our customers we are providing this early version of the manuscript. The manuscript will undergo copyediting, typesetting, and review of the resulting proof before it is published in its final form. Please note that during the production process errors may be discovered which could affect the content, and all legal disclaimers that apply to the journal pertain.



Optimizing Cloud Solutioning Design

Aly Megahed*, Ahmed Nazeem**, Peifeng Yin*, Samir Tata* **, Hamid Reza
Motahari Nezhad**** and Taiga Nakamura*

IBM Research - Almaden,
650 Harry Rd, San Jose, CA 95120, USA
*{aly.megahed,peifengy,taiga}@us.ibm.com, **ahmed.nazeem@ibm.com,
samir.tata@gmail.com, *motahari@ieee.org

Abstract

The economics of the cloud model has been encouraging IT enterprises to migrate from on-premise environments to public, private, or hybrid cloud solutions. To perform such a migration, a cloud offering needs to be chosen and a cloud solution needs to be built. In industrial settings, cloud designers may spend days or even weeks to come up with an acceptable cloud solution with at a low cost/price. Like any manual process, it is obvious that such a cloud solution design process is error prone, time consuming, and does not guarantee an optimal output, e.g. a solution with a minimum cost/price. Different from existing works that solve the problem from the user's angle, we solve it from the cloud provider's prospective, who aims at offering customized cloud solutions for different user requirements at low costs. Such difference requires a unique way of problem modeling. Through analyzing real business data, we abstract the problem into a general attribute-value combinations and formulate a powerful integer programming optimization model to solve it. The general form of the optimization model allows various definitions of customer requirements as well as cloud offerings. Our novel optimization approach for cloud solution design satisfies client requirements, cloud offering constraints, and produces a solution at a minimum cost in a short time, if one exists. We evaluated our solution on realistic data against two baseline approaches. The numerical results show both the effectiveness and efficiency of our approach as well as its practical potential.

Keywords:

Cloud Solution Design; Cloud Computing; Optimization; Integer Programming; Operations Research; Heuristics

1. Introduction

Cloud Computing is gaining momentum in the Information Technology (IT) scope as an emerging computing paradigm for managing and delivering services over the internet [1]. Due to its economic model based on pay-as-you-go plans, IT enterprises are shifting from on-premise environments to public, private, or hybrid clouds. Indeed, cloud environments have presented novel application deployment models offering more convenient costs, high availability, and flexible elasticity.

Over the last few years, the rate of adoption of cloud computing and specifically managed cloud hosting and migration among enterprise customers has actually accelerated significantly. In the enterprise marketplace, managed cloud services are common in which an enterprise migrates some or all its applications from their own data centers to cloud and expects the cloud providers to manage it for them [2].

The problem of hosting workload of an enterprise customer (whether new workload or migration) into an enterprise cloud service provider is a complex and challenging problem. This is because the hosting entails provision of compute resources (e.g., virtual machines), storage, and network, and platform resources to host and run client applications with desired service levels on potentially shared infrastructure components. The managed cloud hosting calls for monitoring and management tools at various levels from infrastructure, compute, storage, networking and application levels to provide transparency and monitoring of such service levels.

The migration job, in addition to new workload hosting, requires analyzing the existing customer workload, and infrastructure needs and configurations and map, upgrade or optimize those in the service provider's provisioning environment which adds to the complexity. Last but not least, the price of cloud

migration or hosting job for a customer, and cost of running such a workload for the service provider, is a key factor and driver of any cloud hosting or migration task.

Any cloud hosting or migration project entails a technical solution design phase. The proposed technical solution to an enterprise customer details the required compute, storage, networking, platform, application, monitoring and management applications and resources along with associated service levels for each. The solution design process starts with capturing and documenting client requirements. The requirements include IT requirements, i.e. the specification of the needed IT resources (at all level of stack from infrastructure to application), or existing client environment (in case of a migration project), and business-level requirements and objectives. While there are multiple business level objectives, the most common is cost saving as the result of migrating to or adopting cloud, as opposed to on-premise data center operation.

In large client cloud hosting or migration projects, solving the solution design problem is a complex and tedious task. This is because given a specific client IT requirement, and different solution components and cloud delivery locations of a service provider, it is often possible to generate multiple solution alternatives. Generating a detailed technical solution design for a cloud project can take a team of IT architects days or weeks (depending on the scope and complexity of requirements). Also, it is not guaranteed that the solution architects can find the optimal (or the best alternative that is available among multiple possible solutions) after such an exercise, both in terms of price for the customer and cost of solution delivery for the client.

The problem that we tackle in this paper is that of automatically computing a cost optimized cloud solution for a given client IT requirements (including the delivery locations, and all levels of infrastructure and applications needs) by finding the optimized combination of solution components, from a cost point of view, offered by a service provider that meets client's requirements. We model IT requirements of a client as a set of constraints expressed over a generic model of functional and non-functional requirements of IT resources and applications.

The solution elements of the service provider are expressed over a generic cloud
 60 IT resources model. In this model, various capabilities are modeled as objects
 with their variations in terms of different acceptable values for any given object's
 attribute. There are also solutioning rules that constrain value selection and
 enforce generation of valid solution combinations.

In the literature, while there are multiple works (e.g., [3, 4, 5]) that come
 65 up with a price optimized solution for a client in leveraging a public cloud
 resources (some by considering the mix of on-demand and reserved resources
 (e.g. [3]), they all tackle the problem from a cloud client or consumer point
 of view. In this paper, we propose a novel cloud solution design approach that
 includes a mathematical optimization model that produces a cloud solution with
 70 an optimal/minimum cost, from the service provider's point of view. The key
 advantage of our method is proposing a generic model for IT requirements and
 cloud offerings and solution elements over which customized client requirements
 can be expressed, and formulate the problem of cost optimized solution design as
 an integer programming problem. We have implemented the proposed approach,
 75 and present the result of experiments and evaluation that shows the practicality
 of the proposed solution.

The rest of paper is structured as follows. Section 2 presents the state
 of the art related to the issue of migrating local IT environments to cloud
 platforms. We then detail our optimization-based approach for cloud solution
 80 design in Section 3. Section 4 presents the implementation of our approach and
 its experimentation with respect to two baseline solution methods that we have
 also implemented. Finally, Section 5 concludes our paper and presents some
 directions for future research.

2. Literature Review

85 Many enterprises choose to migrate their local IT environment to a cloud
 platform due to its advantage on flexible scalability and low cost. The migration
 process also attracts researchers' attention and many works focus on solving

different research issues.

Early works are usually a report or case study, analyzing the whole migration
 90 process of a particular application/environment. In [6], Barker and Shenoy gave
 an empirical study of running latency-sensitive applications in cloud environ-
 ment and reported nearly 75% service quality degrade for disk-bound latency-
 sensitive tasks. In [7], Khajeh-Hosseini et. al. studied the migration of an IT
 system in oil & gas industry in terms of the benefit and risk. In [8, 9], au-
 95 thors reported the experience of migrating Hackstat, an Open Source Software
 (OSS) framework, to the cloud. A more recent work [10] surveyed undergradu-
 ate students to reveal factors impacting end users' switch to cloud.

Considering both benefit and risk, many works develop diversified tools and
 frameworks to facilitate the decision making process. Lewis et. al. [11] developed
 100 SMART methods to help enterprise determine their service functionality when
 migrating to a service-oriented architecture (SOA) cloud. Misra and Mondal [12]
 proposed a Return on Investment (ROI) model to analyze companies's cost and
 benefit when incorporating cloud into their business. Saripalli and Pingali [13]
 apply the multiple attribute decision methodology to help decision making with
 105 respect to different cloud decision objectives. In [14], a general decision pro-
 cess, CloudStep, is presented to support legacy application migration to cloud.
 Other works focus on a particular factor such as cost of deployment [15, 16], net-
 work [17], security and privacy [18, 19, 20]. Particularly in [21], Jamshidi et. al.
 gave a literature review of selected 23 works on cloud migration, summarizing
 110 its major motivations, existing methods and techniques as well as predicting fu-
 ture research dimensions. To bridge migration gap, TOSCA [22] was designed
 to represent the application's topology. Furthermore, Bergmayr et. al. [23]
 developed CAML, a UML-based language to express deployments in cloud.

Besides high-level analysis and theoretical modeling, there are works aiming
 115 at reconfiguration of existing systems when migrated to cloud. In [24], Frey et.
 al. show the unmodified system has scalability issues (either under-provision or
 over-provision) after migrating to cloud. They propose several heuristic rules
 to improve resource efficiency. In [25], a utility function was defined to find

optimal distribution of an application in cloud. Trummer et. al. [4] model the application deployment as a constraint-satisfying optimization problem and rely on the constraint solver to get optimal solution in terms of cost. Aniceto et. al. [3] aim at optimizing the mixture of on-demand and reserved instance in cloud to cover variable computation tasks at minimum cost. In [5], an auto-scaling mechanism is proposed for VM start-up and shut-down activities in order to complete scientific computation tasks within time and budget constraints. Other works adopt diversified techniques such as evolutionary optimization [26, 27, 28], particle swarm optimization [29], multi-goal genetic search algorithm [30], and so on.

In this work, we also focus on the optimization problem. However, different from existing works that solve the problem from the user angle, we solve it from the aspect of a cloud provider, who aims to offer customized cloud solutions for different user requirements at low cost. Such difference requires a unique way of problem modeling. By analyzing real business data, we abstract it into a general attribute-value combination problem and take advantage of powerful integer programming to solve it. The general form of the optimization model allows variant definitions of customer requirements as well as cloud offerings. Evaluation shows promising results and practical potential.

3. Approach for Cloud Solution Design

3.1. Approach Overview

Cloud providers, such as IBM and Amazon, deliver multiple Cloud offerings that provide users and companies with access to an integrated set of managed IT resources including infrastructure, platform and applications. Managed IT infrastructure resources include virtual machines and network. Managed platform resources include middleware and database.

The general process a cloud provider follows to deliver a cloud solution is presented in Figure 1. It consists in three steps: Requirement capturing, Solution design and Delivery specification.

3.1.1. Step 1: Requirement capturing

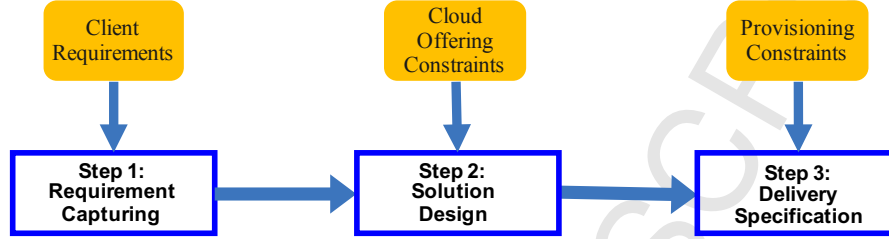


Figure 1: Approach Overview of Cloud Solutioning

The first step in a cloud sales deal is capturing the client requirements including application hosting, infrastructure needs, service level requirements, the need for disaster recovery, database resiliency, backup, etc. This consists in considering a set of attributes that characterize cloud offerings. The requirement consists in a set of constraints on the values those attributes can take. For example, if we consider a VMware offering in IBM Cloud, we may consider a set of attributes to formally describes the offering. These attributes may include:

- the *data center* selected by the client to host his/her workloads,
- the VMware *offering type*, with values such as *vCenter* and *vSphere*,
- the *number of clusters* requested by the client,
- the *number of virtual machines* per cluster to migrate and their characteristics in terms of CPU, RAM and storage,
- the *server size*, with values such as *Small*, *Standard*, *Medium*, and *Large*,
- the *server total cores* of a server,
- the *storage type*, with values such as *VSAN*, *Endurance* and *ISA*,
- the *operation System*, with values such as *AIX*, *RHL* and *Win*,
- the *Disaster Recovery*, with values such as *Yes* and *No*.

- etc.

3.1.2. Step 2: Solution design

The second step, within cloud solutioning, is solution design. This step consists in determining the set of accepted/possible values of the list of attributes that describe the considered cloud offering.

Beside client requirement that define attribute values to be included or to be excluded in the solution, there are cloud offering constraints that should be also satisfied by the solution. There are mainly two types of cloud offering constraints. The first type concerns the constraints that define sets of combinations of attributes and their corresponding values that must be either included together or excluded together in the solution. For example, let's consider the attributes *server size* and *server total cores* with the corresponding values *Small* and *12*. This combination of attributes and values are either included together or excluded together in the solution. The second type of cloud offering constraints concerns constraints that define sets of combinations of attributes and their values are never included together in the solution. For example, not all VMware offerings are available in all possible data centers, not all server sizes are available with all VMware offerings.

3.1.3. Step 3: Delivery specification

The third step, consists in specifying the delivery information necessary to provision the designed cloud solution. This specification includes the point of delivery, or PoD, where the servers will be provisioned, the IP addresses of servers, etc.

3.2. Need for optimization

In the general case, solution design may come up with multiple possible solutions that satisfy client requirements and cloud offering constraints. The difference between these solutions would be the cost/price. In industrial settings, cloud designers may spend approximatively one day to come up with a solution with a reduced cost. It is obvious that such a process is error prone, is time

195 consuming and does not guarantee an optimal solution, e.g. a solution with a minimum cost.

The objective of this paper is to tackle these issues, by providing an optimization model for cloud solution design that satisfy client requirements and cloud offering constraints and producing a solution, if there is any, with a minimum cost in a reasonable time.

3.3. Problem Abstraction and Optimization Model

In this subsection, we first abstract our problem, then we present the notation of our optimization model, and finally we formulate that model.

3.3.1. Problem Abstraction

205 There are multiple solution attributes. We denote the set of solution attributes as the set S . For each solution attribute $s \in S$, there are multiple possible values that can be chosen. However, in the solution to be proposed to the client, only one of these values (or none) is chosen for each attribute. Let the set V_s be the set of possible values for attribute $s \in S$.

210 There is an associated cost codes for multiple combinations of values for a subset of the attributes. Let the set of cost codes be F . Each cost code $f \in F$ has a cost $cost_f$ and the total solution cost is the sum of the costs of all cost codes that are enabled in the solution. A cost code is enabled whenever all of the combinations it is defined upon are included in the solution. For each cost code $f \in F$, we define a set C_f for all combinations of attributes and their values that enable it. That is, an element $(v, s) \in C_f$ is a tuple of the attribute $s \in S$ and its corresponding value $v \in V_s$ that form one of the enablers of that cost code $f \in F$.

220 We define the set NA as the set of combinations of all attributes $s \in S$ and their corresponding values $v \in V_s$ that are not allowed to be in our solution. Similarly, we define the set MI as the set of combinations of attributes $s \in S$ and their corresponding values $v \in V_s$ that must be included in the current solution. Set IT is the set of combinations of attributes and their corresponding values

that must be either included together or excluded together in the solution. That
 225 is, for each element (s, v, s', v') of this set, if solution attribute $s \in S$ and its
 corresponding value $v \in V_s$ is included in the solution, then attribute $s' \in S$,
 with its corresponding value $v' \in V_{s'}$, must be included in the solution, while if
 the former is not included, the latter must be excluded, too. Additionally, set
 IF includes the sets of combinations of pairs of attributes and their values that
 230 enables the following logic: For each element (set) in IF , if all attribute and
 their values, except for the last attribute/value, are included in the solution,
 then that last attribute and its value have to be included in the solution as well.
 Obviously, each element/set in IF has to be ordered (at least its last element
 has to always exist last) to enable this logic. Further, set IFN includes the
 235 forbidden combinations. That is, it contains the sets of combinations of pairs
 of attributes and their values that cannot be included together in the solution.
 Each element in this set is a set of pairs of 2 or more attribute/values.

We note that in any particular problem instance, not all the sets/logic we
 defined above are necessarily included. So, in a problem instance, either none,
 240 some, or all of the sets/logic we discuss here would be applicable. We include
 all of them in our model to be able to handle the most general case, while
 noting that including or excluding any of the constraints below does not affect
 the solution time of our model for realistic instances as we will see in the next
 section below. Table 1 summarizes the sets used in our model. Apart from these
 245 aforementioned sets, the only other parameter/data input to the model is the
 cost code values, $cost_f$, for each cost code $f \in F$.

With the aforementioned dynamics, our problem becomes: which attributes
 and their values should be included in the solution in order to minimize the to-
 tal solution cost while satisfying all the given solution constraints? To solve
 250 this problem, we formulate and solve an integer programming (IP) optimization
 model. We next provide the notation for our model then present its mathemat-
 ical formulation.

Table 1: Sets of Our Optimization Model

Set Name	Set Description
S	Set of solution attributes
V_s	Set of values for solution attribute $s \in S$
F	Set of cost codes
C_f	Set of pairs all attributes and their corresponding values that enable cost code $f \in F$
NA	Set of pairs of all attributes and their corresponding values that are not allowed to be in our solution
MI	Set of paris of all attributes and their corresponding values that must be included in our solution
IT	Set of quintuples (s, v, s', v') of attributes and their corresponding values that must be either included together Or excluded together in the solution, i.e., either (s, v) and (s', v') are included in the solution together, or both are excluded together
IF	Set of combinations of attributes and their corresponding values that are required to follow the following logic: For each element $i \in IF$, where $i = \{(s_1, v_1), \dots, (s_i, v_i)\}$, if all pairs of (attribute, value) in i , except for (s_i, v_i) are included in the solution, then (s_i, v_i) has to be included in it as well. $ i $ is the cardinality of $i \in IF$
IFN	Set of forbidden combinations of (attribute, value) pairs, i.e., combinations of (attributes, value) pairs that cannot be included simultaneously in the solution. Each element i of this set is a set of two or more (attribute, value) pairs $(s, v) : s \in S, v \in V_s$. $ i $ is the cardinality of $i \in IFN$

3.3.2. Model Notation

We define the following three sets of binary decision variables before formulation our optimization model to solve our problem: variable Y_{vs} is 1, if value

$v \in V_s$ for attribute $s \in S$ is included in our solution, and zero, otherwise. Variable X_f is 1, if cost code $f \in F$ is enabled in our solution, and zero, otherwise. Variable $cSlack_s$ is 1 if attribute $s \in S$ is fulfilled in a customized way, and zero otherwise. The reason we defined that latter variable is because not all
 260 combination of attribute-values have a feature code. Such combinations lead to customized solutions with custom and more expensive costs. That is why, typically, solution designers try to use the fewest possible number of these combinations. The way we model this is by adding that binary variable representing each of the customized attribute-value combinations chosen in the optimal solu-
 265 tion, along with a high penalty to minimize the choice of these custom solutions as much as possible. Table 2 summarizes the decision variables of our solution model.

Table 2: Decision Variables of Our Optimization Model

Variable Name	Variable Description
Y_{vs}	$\begin{cases} 1, & \text{if value } v \in V_s \text{ for attribute } s \in S \text{ is included in our} \\ & \text{solution, and} \\ 0 & \text{otherwise} \end{cases}$
$cSlack_s$	$\begin{cases} 1, & \text{attribute } s \in S \text{ is fulfilled in a customized way, and} \\ 0, & \text{otherwise} \end{cases}$
X_f	$\begin{cases} 1, & \text{if cost code } f \in F \text{ is enabled in our solution, and} \\ 0, & \text{otherwise} \end{cases}$

3.3.3. Model Formulation

We now present the formulation of our IP optimization model as follows:

$$\text{Min} \quad \sum_{f \in F} \text{cost}_f \cdot X_f + \text{cutomizationPenalty} * \sum_{s \in S} c\text{Slack}_s \quad (1)$$

$$\text{s.t.} \quad \sum_{v \in V_s} Y_{vs} = 1, \quad \forall s \in S \quad (2)$$

$$X_f \geq \sum_{(v,s) \in C_f} Y_{vs} - (|C_f| - 1), \quad \forall f \in F \quad (3)$$

$$X_f \leq Y_{vs} \quad \forall f \in F, (v, s) \in C_f \quad (4)$$

$$\sum_{f \in F, \exists v: (v,s) \in C_f} X_f + c\text{Slack}_s \geq 1, \quad \forall s \in S \quad (5)$$

$$Y_{vs} = 0, \quad \forall (s, v) \in NA \quad (6)$$

$$Y_{vs} = 1, \quad \forall (s, v) \in MI \quad (7)$$

$$Y_{vs} = Y_{v's'}, \quad \forall (s, v, s', v') \in IT \quad (8)$$

$$\left(\sum_{(s,v) \in i \setminus \{(s_i, v_i)\}} Y_{vs} \right) - (|i| - 2) \leq Y_{v_i s_i}, \quad \forall i \in IF \quad (9)$$

$$\sum_{(s,v) \in i} Y_{vs} \leq |i| - 1, \quad \forall i \in IFN \quad (10)$$

$$X_f \in \{0, 1\}, \quad \forall f \in F \quad (11)$$

$$Y_{vs} \in \{0, 1\}, \quad \forall s \in S, \quad \forall v \in V_s \quad (12)$$

Where objective function 1 minimizes the total cost of the solution, which is the sum of the costs of all cost codes included in that solution in addition to a customization penalty for selecting attribute values that are not part of any selected cost code. Constraint 2 ensures that exactly one value is chosen for each solution attribute. Constraints 3 and 4 set the logic of enabling a cost code. That is, a cost code is enabled if and only if all combinations of attributes and their values, that enable that cost code, are included in the solution. Constraint 5 forces the customization penalty of an attribute to one if no cost code that covers this attribute is enabled in the solution. Constraint 6 ensures that attributes and their values, that are not wanted by the client, are not included in the solution. Similarly, constraint 7 forces attribute values, requested by the client,

to be included in the solution. Constraint 8 ensures that for each quintuples in set IT , the corresponding attributes and values are either included together or excluded together. Constraint 9 guarantees the required logic for set IF , where for each element in that set, if all attribute/value combinations in it, except
285 the last one, are included in the solution, then that last one has to be included there as well. The logic corresponding to set IFN is captured in constraint 10, wherein if value $v \in V_s$ is included in the solution for attribute s , then value $v \in V'_s$ is included in the solution for attribute s' cannot be included in the solution. Lastly, constraints 11 and 12 are the binary constraints for our
290 decision variables.

We end this section noting that our optimization model and approach aims at finding the solution at a minimum cost not the price. Typically, service providers get solutions at the least possible cost, and then they determine the gross profit margin to add to this cost in order to come up with the pricing that
295 will increase the chances of the provider winning business. While we here aim at finding the optimal-cost solution through our approach, finding the optimal pricing is out of scope for this work. For price optimization given the cost, we refer the reader, for instance, to the textbook of Phillips [31].

4. Implementation and Experimentation

300 In this section, we describe the data used in our experimentation that we collected from a real cloud solutioning application of one of the world's largest cloud providers in Section 4.1. We then describe two baseline solution methods for our problem in Section 4.2. Lastly, in Section 4.3, we provide the implementation of our optimization method as well as the two baseline methods, and
305 compare the results of applying all three methods to our realistic data.

4.1. Data Collection

For our experiments, we collected three types of real data from the large cloud service provider for which this work was developed and implemented: i)

attributes and the domain of their possible values for defining solutions, ii) the
 310 logic/constraints that define valid solutions (correlated to NA set), iii) combinations of attribute-value pairs that define feature codes with the corresponding costs (corresponding to MI set), and iv) mandatory (IF) and forbidden attributes (IFN) dependencies.

For the first one, we analyze real business data records and summarize 39
 315 attributes, covering aspects such as data center location, data center type, OS, database type, database size, data recovery services, and so on. Table 3 shows an example of three attributes and their value choices. On average, each attribute has a domain of 9.36 value choices. Thus, there are about $\sim 10^{28}$ combinations in total, which is obviously a massive search space. For ii) and iii), after analyzing
 320 existing costing data, we obtained 209 cost possibilities, of which each cost is associated with 4.44 pairs of solution attribute & value. Table 4 shows three examples of costing rules that are related to OS, data center type and data recovery services. For confidentiality issue, we replace the real cost values with different letters, representing different costs.

325 Finally, for mandatory (IF set) and forbidden attribute (IFN set) dependencies, we obtain 4,383 and 2,161 respectively. The mandatory attribute dependency are sets of attribute-value pairs that purely depend on others. For example, some location only supports one particular data center type. So once the client determines to migrate their IT environment to such location, there
 330 is no choice of data center type. The forbidden attribute dependency are sets of attribute-value pairs that are not allowed together. They are not covered by cost possibilities since they are not directly correlated with cost. One example illegal combination of OS and database type. Some database can only work in a particular OS, e.g., linux. Thus choice of non-linux OS excludes the possibility
 335 of installing such database.

4.2. Baseline Solution Methods

In this section, we describe two baseline methods that we compare to our optimization method in the next section. The first baseline method is simply

Table 3: Example of Attribute Domain

Attribute	Domain
Operation System (OS)	AIX 7, RHL 6, Win 2012
Data Center Type (DCT)	Type I, Type II
Disaster Recovery (DR)	Y, N
CPU cores	16, 32, 64
RAM	64, 128, 256
Service Level	Level I, Level II, Level III

Table 4: Example of Cost

OS	DCT	DR	Cost
Win 2012	Type I	N	x
AIX 7	Type I	Y	y
RHL 6	Type II	Y	y

choosing feasible attribute values at random from every attribute, putting in
 340 consideration not to choose the values that are not allowed to be chosen and
 choosing those that we must choose. Algorithm 1 illustrates this method. Note
 that we use the same notation in this section as the one we used in Section 3.3.1.

In this algorithm, we proceed progressively to randomly pick one attribute
 and one of its feasible values at each iteration. Given the partial solution con-
 345 structed at any point of the algorithm evolution, the set of feasible values of the
 picked attribute is evaluated. If this set of feasible values is empty, we restart
 the algorithm. To calculate the cost of our solution, we iterate over all cost
 codes, check whether a cost code is enabled, and add its cost, if it is. Finally,
 the customization penalty is added for attributes not covered by any of the
 350 enabled feature codes.

The second baseline method that we present next is a greedy heuristic for
 choosing a cheap solution. This heuristic illustrated in Algorithms 2, 3, and

Algorithm 1 Baseline Solution Method 1: Random Selection

```

1:  $RemainingAttributes \leftarrow S$ 
2:  $CurrentSolution \leftarrow \phi$ 
3: while  $RemainingAttributes \neq \phi$  do
4:   Choose a random attribute  $s' \in RemainingAttributes$ 
5:    $VV_{s'} \leftarrow \{v' \in V_{s'} \text{ s.t. the constraints implied by } (NA, MI, IT, IF, IFN) \text{ are satisfied given } CurrentSolution\}$ 
6:   if  $VV_{s'} = \phi$  then
7:      $RemainingAttributes \leftarrow S; CurrentSolution \leftarrow \phi$ 
8:     Continue
9:   Choose a random value  $v'' \in VV_{s'}$ 
10:   $CurrentSolution \leftarrow CurrentSolution \cup \{(v'', s')\}$ 
11:   $RemainingAttributes \leftarrow RemainingAttributes \setminus \{s'\}$ 
12:  $TotalCost \leftarrow 0$ 
13: for Cost code  $f \in F$  do
14:   if  $(\forall (v, s) \in C_f, (v, s) \in CurrentSolution)$  then
15:      $TotalCost \leftarrow TotalCost + cost_f$ 
16:  $TotalCost \leftarrow TotalCost + customizationPenalty * \text{num of attributes not covered by any feature code}$ 

```

4 has 2 precedures:(i) **CustomizeRemainingAttributes** which selects values randomly for the attributes not covered by the given Solution ($CurrSol$), and

355 (ii) **GreedySearch** which is a recursive function that constitutes the core of the algorithm. The validity of the given solution is checked first. If the solution is valid, the algorithm proceeds by trying iteratively to add one feature code at a time and proceeding recursively from this point in a depth first manner. On the other hand, if the given solution is invalid, the algorithm bounds this

360 search path. The order of feature code exploration is in ascending order of their cost. Once a full path of feature codes is explored, the procedure **CustomizeRemainingAttributes** is invoked to get a customized solution for attributes not covered by the feature codes in the solution. Finally, the global variable

SolutionFound is used to exit the algorithm after finding the first feasible solution. Discarding the use of this variable will turn the algorithm into a full enumeration algorithm where all feasible solutions are enumerated.

Note that constraints check in Lines 13 and 30 can be time consuming, and hence it needs to be implemented in an efficient way. To this end, we utilized special problem pertinent structural properties to alleviate the computational complexity involved in these steps. This is, therefore, one of the strong aspects of our optimization model, that we can encode these constraints in a straightforward and simple manner with no deep knowledge of the problem structural properties, while still getting a more efficient solution in terms of both cost and run-time. Further, such optimal solution could be significantly lower than those obtained by the limited baseline methods that we are presenting here, as we will show in the next subsection.

Algorithm 2 Baseline Solution Method 2: Greedy Heuristic - Part 1

```

1: procedure MAIN
2:    $RemCodes \leftarrow F$ 
3:    $RemAttr \leftarrow S$ 
4:    $CurSol \leftarrow \phi$ 
5:    $TotCost \leftarrow 0$ 
6:   global :  $SolutionFound \leftarrow \text{False}$ 
7:   Sort the cost codes in set  $F$  in ascending order of cost in  $cost_f$ 
8:    $(TotalCost, Solution) \leftarrow \mathbf{GreedySearch}(CurSol, RemCodes,$ 
       $RemAttr, TotCost)$ 

```

4.3. Implementation, Experiments, and Practical Implications

We implemented our optimization method as well as the two baseline methods. All implementations were done in the *python* programming language. We solved the optimization model using the commercial solver CPLEX [32]. We then constructed some experiments to examine the performance of our optimization method compared to the two baseline method.

Algorithm 3 Baseline Solution Method 2: Greedy Heuristic - Part 2

```

1: procedure GREEDYSEARCH(CurrSol, RemCodes, RemAttr, TotCost)
2:   if global:solutionFound then
3:     return
4:   if CheckConstraints(CurrSol, NA, MI, IT, IF, IFN) = False then
5:     return
6:   for  $f \in \text{RemCodes}$  (where we iterate on the sorted set  $F$ ) do
7:      $\text{CurrSol}' \leftarrow \text{CurrSol} \cup \{(v, s) \in C_f\}$ 
8:      $\text{RemCodes}' \leftarrow \text{RemCodes} \setminus \{f\}$ 
9:      $\text{RemAttr}' \leftarrow \text{RemAttr} \setminus \{s \in C_f\}$ 
10:     $\text{TotCost}' \leftarrow \text{TotCost} + \text{cost}_f$ 
11:    GreedySearch(CurrSol', RemCodes', RemAttr', TotCost')
12:  global:solutionFound  $\leftarrow$  True
13:  return CustomizeRemainingAttributes(RemAttr, TotCost, CurrSol)

```

We implemented the optimization model using the CPLEX 12.7 library for Python 3.5. More specifically, we used the function `cplex.Cplex.variables.add` to add the model variables, the function `cplex.Cplex().linear_constraints.add` to add the model constraints, and the function `cplex.Cplex().objective.set_linear` to specify the model objective function, and finally `cplex.Cplex().model.solve` to solve the optimization model.

We tried multiple settings to improve the optimization model performance, but none of them proved to be significantly different from the default settings. In particular, we tried adding cuts, changing the LP algorithm, solution polishing, changing the node selection criteria, changing the pivoting criteria, and changing the branching criteria. But since none of these changes proved to provide statistically different solutions, we ended up using the default CPLEX settings.

The objective of these experiments is threefold. First, we aim at validating the optimality of the solutions gotten from our optimization approach. That is,

Algorithm 4 Baseline Solution Method 2: Greedy Heuristic - Part 3

```

1: procedure CUSTOMIZEREMAININGATTRIBUTES(RemAttr, TotCost, CurrSol)
2:   RemAttr'  $\leftarrow$  RemAttr
3:   TotCost'  $\leftarrow$  TotCost
4:   CurrSol'  $\leftarrow$  CurrSol
5:   while RemAttr'  $\neq \phi$  do
6:     Choose a random attribute  $s' \in \text{RemAttr}'$ 
7:      $VV_{s'} \leftarrow \{v' \in V_{s'} \text{ s.t. the constraints implied by } (NA, MI, IT, IF, IFN) \text{ are satisfied given } CurrSol'\}$ 
8:     if  $VV_{s'} = \phi$  then
9:       RemAttr'  $\leftarrow$  RemAttr
10:      TotCost'  $\leftarrow$  TotCost
11:      CurrSol'  $\leftarrow$  CurrSol
12:      Continue
13:     Choose a random value  $v'' \in VV_{s'}$ 
14:     CurrSol'  $\leftarrow$  CurrSol'  $\cup \{(v'', s')\}$ 
15:     RemAttr'  $\leftarrow$  RemAttr'  $\setminus \{s'\}$ 
16:     TotCost'  $\leftarrow$  TotCost' + CustomizationPenalty
   return TotCost', CurrSol'

```

these solutions cannot be worse than those of the baseline methods for instance, and it should be equal to the solution found by the brute-force enumeration.

400 Second, we compare the efficiency of our optimization approach compared to the two baseline methods. This is done via comparing the solution times of each method. Lastly, the effectiveness of each method is compared through calculating the relative increase in cost for each baseline method compared to the optimal (minimal) cost.

405 We explain our experimental design as follows: We constructed approximately 1000 problem instances using our aforementioned realistic data for Cloud solutioning attributes and their values, where in each instance, we restrict the values that can be chosen for each attribute to a random subset of these val-

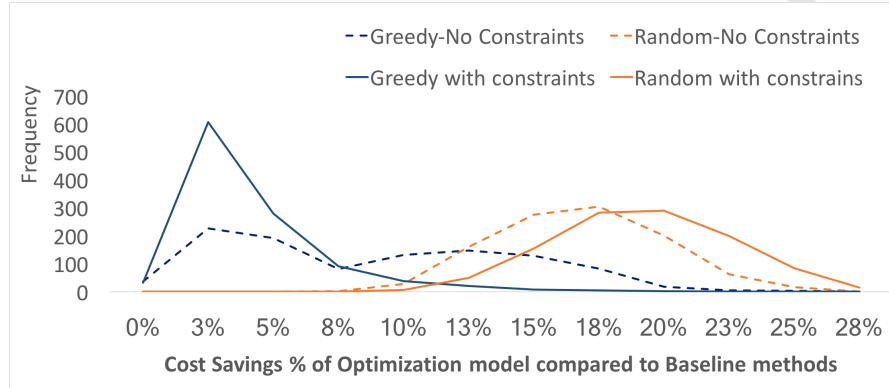


Figure 2: Histogram of Cost Savings % obtained by Our Optimization Method over Baseline methods

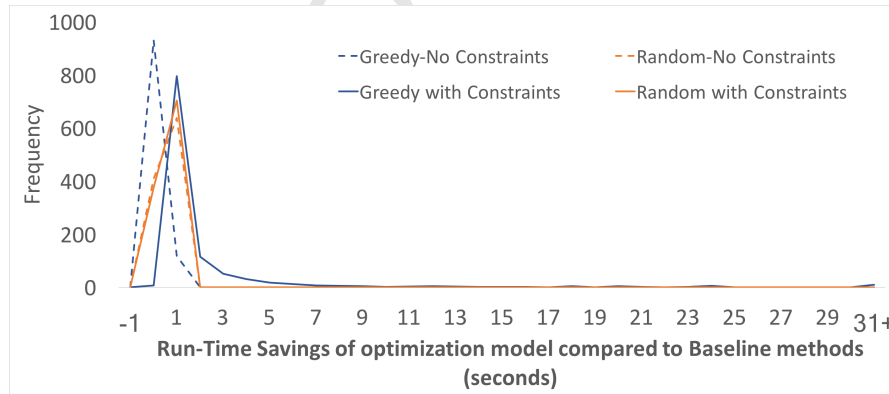


Figure 3: Histogram of Run-Time Savings obtained by Our Optimization Method over Baseline methods

ues. Additionally, we used a realistic embodiment for the constraints implied
 410 IF and IFN . We solved all the instances with the constraints implied by (IF ,
 IFN) and without these constraints. All of our experiments were run on a 2.1
 GHz 24-core Intel(R) Xeon(R) CPU E5-2683 v4 processor with 64 MB of cache
 memory and 256 GB of RAM.

We found that our optimization model is robust; it solves each of these
 415 instances in less than 1 second. We also found that the optimal cost could be
 significantly lower than that obtained by each of the two baseline methods.

Figures 2 and 3 show the distributions of the cost savings and the run time
 savings of the optimization method over both baseline methods (the greedy
 heuristic and the random selection heuristic) with and without the constraints
 420 implied by (IF , IFN). From the figures, we can note that following:

- Comparing the Optimization Model to the Greedy Heuristic:
 - The optimization model cost is significantly lower than the greedy
 heuristic cost.
 - The absence of the constraints implied by (IF , IFN) amplifies the
 425 cost savings obtained by the optimization model.
 - In the absence of the constraints implied by (IF , IFN), the opti-
 mization model is slightly slower than the greedy heuristic.
 - In the existence of the constraints implied by (IF , IFN), the opti-
 mization model is faster than the greedy heuristic.
- 430 • Comparing the Optimization Model to the Random Selection Heuristic:
 - The optimization model cost is significantly lower than the random
 selection cost.
 - The optimization model is slightly faster than the random selection
 cost.
 - 435 – The constraints implied by (IF , IFN) has no significant effect on
 the relative performance of the random selection method in terms of

the cost. However, it increases the variability of solution time of the random selection method.

Tables 5 and 6 summarize the statistics of these results. Looking at these
 440 results, one can obviously tell the effectiveness of our optimization method given
 how huge of a cost saving it could provide for the solution provider when used instead of any of the baseline methods. Also, given how fast our model gets solved in, the efficiency of our method becomes clear, especially with the existence of constraints on the attributes dependencies which are very common in practice.
 445 Note that entries with negative values in Table 6 mean that the optimization model is slower than the corresponding baseline method for this entry.

Finally, we would like to mention that for the sake of validation, we solved the enumerated instances using brute-force enumeration. As expected, the optimal cost using brute-force enumeration is equal to that obtained using our optimization model, and also as expected, the average run time increase compared to
 450 the optimization model is about 62K folds.

There are multiple practical implications to our approach. Since our model runs very quickly (in seconds), it can be easily deployed in a solutioning tool, like the web-based application developed for the cloud service provider for this
 455 this work was done. It also enables users to be able to change any parameters of the problem inputs, and re-run the model to get the updated solution in seconds.

Also, the effort of deploying our optimization model compared to that for heuristic solutions (e.g., the two baseline methods we presented) or manual
 460 solutions is very similar, since any method will just be implemented in the backend of the application and then deployed in production. This is confirmed since we implemented the algorithms for all these methods for the sake of the experiments above.

Moreover, our approach/model also uses the same inputs as the other methods and thus, does not incorporate any additional effort in that regard. Finally,
 465 it has been shown that our model scales much better in the existence of inter-

dependency constraints between attributes.

Table 5: Cost Savings Statistics Comparing Our Optimization Method versus the Two Baseline Methods

	% Cost Decrease of Our Optimization Method over Greedy Heuristic		% Cost Decrease of Our Optimization Method over Random Heuristic	
	With Con- straints	No Con- straints	With Con- straints	No Con- straints
Minimum	0	0	9%	6%
Maximum	20%	23%	27%	24%
Average	3%	8%	18%	16%
Standard Deviation	3%	5%	3%	3%

5. Conclusions and Future Work

We tackled in this paper the issue of cost optimization of a cloud solution for a given client IT requirements. Our contribution consists of a novel approach that incorporates finding the optimized combination of solution components, from a cost point of view, offered by a service provider that meets client's requirements.

We have implemented our optimization method as well as two baseline methods and compared the results of applying all three methods on realistic data. Experimentation results show the effectiveness of our optimization method since

Table 6: Cost Savings Statistics Comparing Our Optimization Method versus the Two Baseline Methods

	% Run-Time Decrease of Our Optimization Method over Greedy Heuristic (in Seconds)		% Run-Time Decrease of Our Optimization Method over Random Heuristic (in Seconds)	
	With Constraints	No Constraints	With Constraints	No Constraints
Minimum	-0.19	-0.88	-0.35	-0.6
Maximum	108	0.05	0.57	0.9
Average	1.57	-0.17	0.04	0.05
Standard Deviation	5.16	0.16	0.1	0.19

we have shown that it provides massive cost savings for the solution provider compared to the baseline methods.

In addition, our experimentation shows the efficiency of our approach as it gets executed in a few seconds at most, compared to the minutes of the baseline methods and the hours (to few days) it used to take human solutioners to get a feasible solution for the considered problem. We have also shown that a brute-force solution to our problem is not applicable at all as it takes hours for realistic-sized instances.

Given the aforementioned effectiveness and efficiency of our approach, as well as the fact that the effort of implementing and deploying our optimization approach compared to the other methods is similar, it is shown to be quite useful, applicable, and impactful for real service providers.

We solved the problem of solution cost optimization, in which we were trying
 490 to find the minimum possible solution that satisfied the client requirements.
 After solution providers solve such costing minimization problem, they need to
 "price" their solution. That is, they need to add some gross profit on top of the
 cost in order to reach the price that they will offer to clients. Obviously, the
 higher the price, the lower their chances of selling their solution versus other
 495 competitors. Thus, a research question, that is a natural extension of our work,
 is: what is the optimal price (or added gross profit on top of the optimal cost)
 that would increase the chance of successfully winning the deal of selling the
 cloud service to clients? Therefore, applying some of the pricing methods in
 the literature of revenue management will be an interesting research direction
 500 to this work.

References

- [1] G. Pallis, Cloud computing: the new frontier of internet computing, IEEE
 internet computing 14 (5) (2010) 70–73.
- [2] D. Linthicum, The case for managed service providers in your cloud strat-
 505 egy, [http://www.infoworld.com/article/2923441/cloud-computing/the-](http://www.infoworld.com/article/2923441/cloud-computing/the-case-for-managed-service-providers-in-your-cloud-strategy.html)
[case-for-managed-service-providers-in-your-cloud-strategy.html](http://www.infoworld.com/article/2923441/cloud-computing/the-case-for-managed-service-providers-in-your-cloud-strategy.html), [Online;
 accessed June 1, 2017] (May 19, 2015).
- [3] I. San Aniceto, R. Moreno-Vozmediano, R. S. Montero, I. M. Llorente,
 Cloud capacity reservation for optimal service deployment, in: Second In-
 510 ternational Conference on Cloud Computing, GRIDs, and Virtualization,
 2011, pp. 52–59.
- [4] I. Trummer, F. Leymann, R. Mietzner, W. Binder, Cost-optimal out-
 sourcing of applications into the clouds, in: Cloud Computing Technology
 and Science (CloudCom), 2010 IEEE Second International Conference on,
 515 IEEE, 2010, pp. 135–142.

- [5] M. Mao, J. Li, M. Humphrey, Cloud auto-scaling with deadline and budget constraints, in: Grid Computing (GRID), 2010 11th IEEE/ACM International Conference on, IEEE, 2010, pp. 41–48.
- [6] S. K. Barker, P. Shenoy, Empirical evaluation of latency-sensitive application performance in the cloud, in: Proceedings of the first annual ACM SIGMM conference on Multimedia systems, ACM, 2010, pp. 35–46.
- [7] A. Khajeh-Hosseini, D. Greenwood, I. Sommerville, Cloud migration: A case study of migrating an enterprise IT system to IaaS, in: Cloud Computing (CLOUD), 2010 IEEE 3rd International Conference on, IEEE, 2010, pp. 450–457.
- [8] M. A. Chauhan, M. A. Babar, Migrating service-oriented system to cloud computing: An experience report, in: Cloud Computing (CLOUD), 2011 IEEE International Conference on, IEEE, 2011, pp. 404–411.
- [9] M. A. Babar, M. A. Chauhan, A tale of migration to cloud computing for sharing experiences and observations, in: Proceedings of the 2nd international workshop on software engineering for cloud computing, ACM, 2011, pp. 50–56.
- [10] S. C. Park, S. Y. Ryoo, An empirical investigation of end-users' switching toward cloud computing: A two factor theory perspective, Computers in Human Behavior 29 (1) (2013) 160–170.
- [11] G. Lewis, E. Morris, D. Smith, Service-oriented migration and reuse technique (smart), in: Software Technology and Engineering Practice, 2005. 13th IEEE International Workshop on, IEEE, 2005, pp. 222–229.
- [12] S. C. Misra, A. Mondal, Identification of a company's suitability for the adoption of cloud computing and modelling its corresponding return on investment, Mathematical and Computer Modelling 53 (3) (2011) 504–521.

- [13] P. Saripalli, G. Pingali, Madmac: Multiple attribute decision methodology for adoption of clouds, in: Cloud Computing (CLOUD), 2011 IEEE International Conference on, IEEE, 2011, pp. 316–323.
- 545 [14] P. V. Beserra, A. Camara, R. Ximenes, A. B. Albuquerque, N. C. Mendonça, Cloudstep: A step-by-step decision process to support legacy application migration to the cloud, in: Maintenance and Evolution of Service-Oriented and Cloud-Based Systems (MESOCA), 2012 IEEE 6th International Workshop on the, IEEE, 2012, pp. 7–16.
- 550 [15] A. Khajeh-Hosseini, I. Sommerville, J. Bogaerts, P. Teregowda, Decision support tools for cloud migration in the enterprise, in: Cloud Computing (CLOUD), 2011 IEEE International Conference on, IEEE, 2011, pp. 541–548.
- 555 [16] A. Khajeh-Hosseini, D. Greenwood, J. W. Smith, I. Sommerville, The cloud adoption toolkit: supporting cloud adoption decisions in the enterprise, *Software: Practice and Experience* 42 (4) (2012) 447–465.
- [17] E. Ahmed, A. Akhunzada, M. Whaiduzzaman, A. Gani, S. H. Ab Hamid, R. Buyya, Network-centric performance analysis of runtime application migration in mobile cloud computing, *Simulation Modelling Practice and Theory* 50 (2015) 42–56.
- 560 [18] H. Mouratidis, S. Islam, C. Kalloniatis, S. Gritzalis, A framework to support selection of cloud providers based on security and privacy requirements, *Journal of Systems and Software* 86 (9) (2013) 2276–2293.
- [19] M. Pavlidis, H. Mouratidis, C. Kalloniatis, S. Islam, S. Gritzalis, Trustworthy selection of cloud providers based on security and privacy requirements: Justifying trust assumptions, in: International Conference on Trust, Privacy and Security in Digital Business, Springer, 2013, pp. 185–198.
- 565 [20] H. Ma, Z. Hu, K. Li, H. Zhang, Toward trustworthy cloud service selection:

- a time-aware approach using interval neutrosophic set, *Journal of Parallel and Distributed Computing* 96 (2016) 75–94.
- [21] P. Jamshidi, A. Ahmad, C. Pahl, Cloud migration research: a systematic review, *IEEE Transactions on Cloud Computing* 1 (2) (2013) 142–157.
- [22] T. Binz, U. Breitenbücher, O. Kopp, F. Leymann, Tosca: Portable automated deployment and management of cloud applications, in: *Advanced Web Services*, Springer, 2014, pp. 527–549.
- [23] A. Bergmayr, J. Troya, P. Neubauer, M. Wimmer, G. Kappel, UML-based cloud application modeling with libraries, profiles, and templates, in: *CloudMDE@ MoDELS*, 2014, pp. 56–65.
- [24] S. Frey, W. Hasselbring, The cloudmig approach: Model-based migration of software systems to cloud-optimized applications, *International Journal on Advances in Software* 4 (3 and 4) (2011) 342–353.
- [25] V. Andrikopoulos, S. G. Sáez, F. Leymann, J. Wettinger, Optimal distribution of applications in the cloud, in: *International Conference on Advanced Information Systems Engineering*, Springer, 2014, pp. 75–90.
- [26] M. J. Csorba, H. Meling, P. E. Heegaard, Ant system for service deployment in private and public clouds, in: *Proceedings of the 2nd workshop on Bio-inspired algorithms for distributed systems*, ACM, 2010, pp. 19–28.
- [27] H. Wada, J. Suzuki, Y. Yamano, K. Oba, Evolutionary deployment optimization for service-oriented clouds, *Software: Practice and Experience* 41 (5) (2011) 469–493.
- [28] Z. I. M. Yusoh, M. Tang, Composite saas placement and resource optimization in cloud computing using evolutionary algorithms, in: *Cloud Computing (CLOUD)*, 2012 IEEE 5th International Conference on, IEEE, 2012, pp. 590–597.

- 595 [29] S. Pandey, L. Wu, S. M. Guru, R. Buyya, A particle swarm optimization-based heuristic for scheduling workflow applications in cloud computing environments, in: Advanced information networking and applications (AINA), 2010 24th IEEE international conference on, IEEE, 2010, pp. 400–407.
- 600 [30] S. Frey, F. Fittkau, W. Hasselbring, Search-based genetic optimization for deployment and reconfiguration of software in the cloud, in: Proceedings of the 2013 International Conference on Software Engineering, IEEE Press, 2013, pp. 512–521.
- [31] R. L. Phillips, Pricing and revenue optimization, Stanford University Press, 2005.
- 605 [32] IBM ILOG CPLEX, V12. 1: User's manual for CPLEX, International Business Machines Corporation 46 (53) (2009) 157.

Authors' Bibliographies

Aly Megahed

Aly Megahed is a research staff member at IBM's Almaden Research Center in San Jose, CA. His current research interests span over building analytical tools for complex service engagements, cloud computing, and IoT, and advancing research in analytics, machine learning, and operations research. Dr. Megahed got his Ph.D. in Industrial Engineering from Georgia Tech. He has done multiple analytical research/consultancy projects for 6 companies in the past and has his work published in several academic journals and conferences, in addition to filing multiple patent disclosures and winning multiple IBM internal awards as well as external ones.

Ahmed Nazeem

Ahmed Nazeem is a research staff member at IBM's Almaden Research Center in San Jose, CA. In his current job, he develops analytical tools for complex service engagements, cloud computing, and IoT, and advances research in cognitive computing, machine learning, and operations research. Dr. Nazeem got his Ph.D. in Industrial Engineering from Georgia Tech. He has his work published in several academic journals and conferences, in addition to filing multiple patent disclosures.

Peifeng Yin

Peifeng Yin is a Research Staff Member at IBM's Almaden Research Center in San Jose, CA. His research interests include service data analytics, machine learning, data mining, and their applications in different areas. Dr. Yin got his Ph.D. in Computer Science and Engineering from Pennsylvania State University. He has his works published in several conferences and journals, in addition to filing multiple patent disclosures.

Samir Tata

Samir Tata is a Research Staff Member at IBM's Almaden Research Center in San Jose, CA. Before joining IBM, he was a Professor at Telecom SudParis, part of Institut Mines-Telecom, France. His current research interests include service oriented computing, business process management, and their applications in virtual enterprises, cloud environments, and the Internet of Things. He was chair of several international conference and workshops. He was/is member of the steering or program committee of several international conferences.

Hamid Reza Motahari Nezhad

Hamid Reza Motahari Nezhad is a Research Group Lead for Cognitive Services, and Member of IBM Academy of Technology at IBM Almaden Research Center. His research interests include areas of data analytics, cognitive computing (NLP, text analytics and machine learning), cognitive business process management, and cognitive enterprise services. He also holds a Visiting Principal Research Fellow position with The University of New South Wales (UNSW), Australia, where he is co-supervisor of Master's and PhD students in computer science, machine learning, and cognitive computing. He has published more than 100 scholarly papers in conferences and journals. He is a senior member of IEEE and a member of ACM. Contact him at motahari@ieee.org

Taiga Nakamura

Taiga Nakamura is a Research Staff Member and Research Manager at IBM's Almaden Research Center in San Jose, CA, where he currently leads the Cloud Services Analytics Research group. He is conducting research on various aspects of solution design for Services and Software, in the area of cognitive and model-based solutioning, cloud services optimization, solution competitiveness, requirements and knowledge management, and quality analysis. Dr. Nakamura received his PhD in Computer Science from the University of Maryland, College Park. He has authored or coauthored more than 30 technical papers and articles, coauthored one book chapter, and have many patents filed and issued. He is a member of ACM, IEEE, and a senior member of IPSJ.

Authors' PicturesAly MegahedAhmed Nazeem:Peifeng Yin

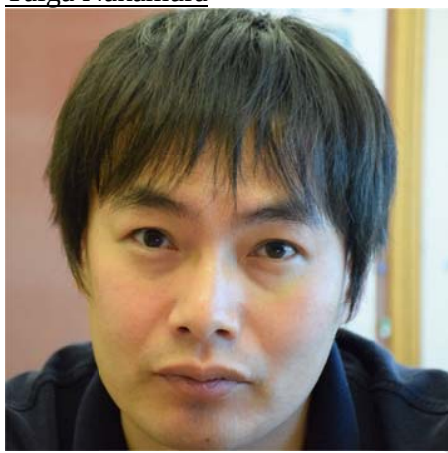
Samir Tata



Hamid Reza Motahari Nezhad



Taiga Nakamura



Highlights

- We tackle the problem of cloud solution design.
- We formulate an optimization model that find lowest cost solution designs.
- We compare our approach to two baseline heuristics as well as a brute force method.
- Numerical results show the efficiency and effectiveness of our approach.